

CS 3308: Information Retrieval

Programming Assignment: Unit 5

Instructor: Rupali Memane

Day of Submission: 2025/12/18

Question of Assignment:

In the previous two development assignments, we have created a process to generate an inverted index based on the corpus posted in the Assignment Files section of Unit 2. In the index part 1 assignment, we created a process that scans a document corpus and creates an inverted index. In the index part 2 assignment, we extended the functionality of the index process by incorporating functionality to:

- Ignore (not include the index) a list of stop words

Edit tokens as follows:

- Terms under 2 characters in length are not indexed
- Terms that contain only numbers are not indexed
- Terms that begin with a punctuation character are not indexed
- Integrate the Porter Stemmer code into our index
- Calculate the tf-idf_{t,d} value for each unique combination of document and term

In this assignment, we will be using the inverted index that was created by the process developed as part of the index part 2 assignment. In unit 4, we learned about the weighting of terms to improve the relevance of documents found when searching the inverted index. First we learned about calculating the inverse document frequency and from this the tf-idf_{t,d} weight. Using the tf-idf_{t,d} weight, we were able to calculate both document and query vectors and evaluate them using the formula for cosine similarity.

In the development assignment for this unit, we will implement these concepts to create a search engine. Your assignment will be to create a search engine that will allow the user to enter a query of terms that will be processed as a ‘bag of words’ query.

Your search engine must meet the following requirements:

- It must prompt the user to enter a query as a ‘bag of words’ where multiple terms can be entered separated by a space.
- For each query term entered, your process must determine the tf-idf_{t,d} weight as described in Unit 4.
- Using the query terms, your process must search for each document that contains each of the query terms.
- For each document that contains all of the search terms, your process must calculate the cosine similarity between the query and the document.
- The list of cosine similarity scores must be sorted in descending order from the most similar to the least similar.

Finally, your search process must print out the top 20 documents (or as many as are returned by the search if there are fewer than 20), listing the following statistics for each:

- The document file name
- The cosine similarity score for the document
- The total number of items that were retrieved as candidates (you will only print out the top 20 documents)
‘Simpson algorithm’ is provided in the output of the search for terms

Assessment:

Your submission will be assessed using the following Aspects. These items will be used in the grading rubric for this assignment. Make sure that you have addressed each item in your assignment.

Was the source code (.py file) for the solution submitted as an attachment to the assignment? (25%)

Was the output of the search for the term ‘home mortgage’ provided in the posting? (25%)

Did the output include the following information, and were there 20 items or fewer listed in the output? (50%)

Output for “home mortgage provided 0 results; here are the results:

```
=====
REQUIRED TEST QUERY: 'home mortgage'
=====
=====
Searching for: 'home mortgage'
=====
[DEBUG] Raw query: 'home mortgage'
[DEBUG] Term 'home' -> stemmed 'home'
[DEBUG] Term 'mortgage' -> stemmed 'mortgage'
[DEBUG] Final processed terms: ['home', 'mortgage']
[DEBUG] Term 'home' not found in index and no similar terms
[DEBUG] Term 'mortgage' not found in index and no similar terms
[WARNING] No valid terms found in index

[INFO] No documents contain ALL terms: ['home', 'mortgage']
[DEBUG] Fallback: Found 0 documents with ANY term

Found 0 candidate document(s)
```

↑
↓
↺
↻
🖨
🗑

NO RESULTS FOUND

=====

Total candidate documents retrieved: 0

Simpson algorithm

=====

TRYING ALTERNATIVE QUERIES

=====

Trying: 'computer'

=====

Searching for: 'computer'

=====

[DEBUG] Raw query: 'computer'

[DEBUG] Term 'computer' -> stemmed 'computer'

[DEBUG] Final processed terms: ['computer']

[DEBUG] Term 'computer' found in index (ID: 51, df: 93)

[DEBUG] Executing SQL for terms: ['computer']

[DEBUG] Found 93 documents with ALL terms

Found 93 candidate document(s)

```
=====
TOP 10 RESULTS (Sorted by Cosine Similarity)
=====
1. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0002.html
   Cosine Similarity: 1.000000

2. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0004.html
   Cosine Similarity: 1.000000

3. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0006.html
   Cosine Similarity: 1.000000

4. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0007.html
   Cosine Similarity: 1.000000

5. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0010.html
   Cosine Similarity: 1.000000

6. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0013.html
   Cosine Similarity: 1.000000

7. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0019.html
   Cosine Similarity: 1.000000

8. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0022.html
   Cosine Similarity: 1.000000

9. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0023.html
   Cosine Similarity: 1.000000
```

```
10. Document: C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm\CACM-0037.html
    Cosine Similarity: 1.000000

Total candidate documents retrieved: 93
Simpson algorithm
```